

DBMS Assignment 10

Instructions:

- Submit one .pdf
- The pdf file should contain the question, query/code, outputs, and explanation.
- The name should be <roll no>_Assignment10.pdf
- Complete the exercise before 4.45 PM and submit the files in Moodle.
- Marks Distribution for both Postgres and Python (**query/code + output+explanation**)

Total Marks = 40

8/04/2025

Note: *Show sufficient use cases and explanations during the evaluation and submission*
Use the dvdrental database

Q1) The marketing team often needs to generate reports that list all films sorted by their rental_rate (both in ascending and descending order). However, retrieving sorted results has become slow due to the growing size of the film table.

1. Create an index to improve the performance of sorting queries on the rental_rate column. **(2)**
Explain how and why this index improves the query performance.
2. Use EXPLAIN ANALYZE to compare the execution time of retrieving sorted film records before and after applying the index. **(2)**
3. Without the index, what internal sort strategy does PostgreSQL use to retrieve sorted results? **(1)**

Q2) A) The business requires that no two customers should have the same email address, and customer email is frequently used for login verification.

- a) Modify the schema or implement a constraint to enforce this business rule. **(1)**
- b) Write a query to retrieve customer information based on their email address. **(1)**
- c) Explain how your solution ensures both **data correctness** and **query (1) performance**.

B) Customer service often runs the following report to find all payments made on a specific date (ignoring time). However, the current query is slow due to the size of the payment table:

SELECT * FROM payment WHERE DATE(payment_date) = '2007-02-15';

- a) Identify the performance issue in the above query and implement a solution to improve it without changing the query syntax. **(1)**
- b) Use EXPLAIN ANALYZE to compare the query's execution time before and after your optimization. **(1)**
- C) Explain how your solution improves query performance. **(1)**

Q3) Retrieve a list of payments made by staff member 2 during the year 2007, where each payment exceeds \$5.00, and display the results ordered by payment date. Then, from the following composite index options for the payment table, choose the best option to optimize the query performance and provide your reasoning:

- **Index Option 1:** (payment_date, staff_id, amount)
- **Index Option 2:** (amount, staff_id, payment_date)
- **Index Option 3:** (staff_id, payment_date, amount)

Additionally, execute the query both without any composite index and with your chosen composite index, and analyze the execution timing for both scenarios. Explain how the chosen index improves performance compared to not using a composite index. **(3)**

Q4) Use the following query:

```
SELECT release_year, length
FROM film
WHERE release_year = 2006 AND length > 100;
```

1. Run the query using EXPLAIN ANALYZE without any index and note the execution plan. **(1)**
2. Create a B-tree index on release_year. Run the query again and check whether the index is used and if a heap fetch is required. **(1)**
3. Create a Covering Index using release_year and length on the film table. Run the query again and identify the type of scan used. **(1)**
4. Briefly explain the difference between a regular index and a covering index, and the role of the INCLUDE clause. **(2)**

Q5) Fetch the film details of the film with length as 116, OR PG rated, OR replacement cost greater than 10, using the film table. Compare the performance of a multi-column index and 3 single column indexes on the query. Reason about your observations. **(4)**

Q6) Fetch the film details of the film with length as 116, PG rated, and replacement cost greater than 10 using the film table.

- a) Create a multi-column index on the search keys to make the query efficient. Compare the execution time of the above query with and without using the index **(3)**
- b) Change the order of the 'AND' clauses in the query and check if it changes how the index is being used **(2)**
- c) Now write queries for the following and examine if the index is being used or not:
 - i) film details of the film with length 116, PG rated **(1 + 1)**
 - ii) film details of the film PG rated, replacement cost > 10 **(1 + 1)**
 - iii) film details of the film with length 116, replacement cost > 10 **(1 + 1)**
- d) Change the order of creation of the multi-column index (in any one way) and showcase the change in the execution times encountered for cases in c) **(2)**

Q7) For the initial query in Q6, construct 3 single-column indexes that can be used and compare performance with the multi-column index used. **(4)**